

Explain Kth Pair Sum

Absolutely. This is a classic **binary search on the answer** problem. The key is that we **never actually construct C**, because $N \times M$ can be as large as 10^{12} .

1. Understanding the question

We are given:

- Array **A** of size **N**
- Array **B** of size **M**
- Integer **K**

We have to consider **every possible pair**:

$$C = \{A[i] + B[j]\}$$

So there are exactly:

$$N \times M$$

elements in **C**.

Then sort **C** and find the **K-th smallest element**.

Example

A = [1, 2, 3]
B = [4, 5, 6]
K = 6

All combinations:

1+4 = 5
1+5 = 6
1+6 = 7

2+4 = 6
2+5 = 7
2+6 = 8

3+4 = 7
3+5 = 8
3+6 = 9

Therefore:

C = [5, 6, 7, 6, 7, 8, 7, 8, 9]

Sorted:

[5, 6, 6, 7, 7, 7, 8, 8, 9]

The 6th smallest is:

7

2. Why can't we simply construct C?

Suppose:

N = 1,000,000
M = 1,000,000

Then:

$$N \times M = 10^{12}$$

That means **C** contains **1 trillion elements**.

Clearly, we cannot:

```
vector<long long> C;
for (...) {
    for (...) {
        C.push_back(A[i] + B[j]);
    }
}
```

This is impossible.

So we need a completely different idea.

3. First observation: We don't actually need C

Think about what the problem is asking.

It asks:

What is the K-th smallest value?

Instead of constructing all values, let's ask a different question:

For some value x , how many pairs (i, j) satisfy $A[i] + B[j] \leq x$?

Let that number be:

$$\text{count}(X)$$

For example:

A = [1, 2, 3]
B = [4, 5, 6]

Suppose:

X = 6

Which sums are ≤ 6 ?

5, 6, 6

So:

$\text{count}(6) = 3$

Now suppose:

X = 7

Sums ≤ 7 are:

5, 6, 7, 6, 7, 7

So:

$\text{count}(7) = 6$

And because $K = 6$, the answer is 7.

This gives us our key observation.

4. The important observation

Suppose the answer is x .

Then there must be at least K elements in C that are $\leq x$.

Therefore:

$$\text{count}(X) \geq K$$

For our example:

$\text{count}(6) = 3 < 6$

So 6 cannot be the answer.

```
count(7) = 6 >= 6
```

So 7 can be the answer.

5. This gives us a monotonic property

This is the most important part.

Consider:

```
X = 5
X = 6
X = 7
X = 8
X = 9
```

The number of sums $\leq x$ can only increase.

For our example:

X	count(X)

5	1
6	3
7	6
8	8
9	9

Notice:

```
1 -> 3 -> 6 -> 8 -> 9
```

It **never decreases**.

Therefore we can binary search for the smallest x such that:

$$\text{count}(X) \geq K$$

That x is exactly the K -th smallest element.

6. So the problem becomes two smaller problems

We need to solve:

Problem 1

For a given x , efficiently calculate:

```
How many pairs have A[i] + B[j] <= X?
```

Problem 2

Binary search the smallest x for which:

```
count(X) >= K
```

Let's solve them one by one.

7. How do we calculate count(X)?

Suppose:

```
A = [1, 2, 3]
B = [4, 5, 6]
X = 7
```

For every $A[i]$, we need:

$$A[i] + B[j] \leq X$$

Rearrange:

$$B[j] \leq X - A[i]$$

This is the key transformation.

For A[i] = 1

We need:

```
B[j] <= 7 - 1  
B[j] <= 6
```

B is:

```
[4,5,6]
```

All 3 satisfy it.

So:

```
3 pairs
```

For A[i] = 2

We need:

```
B[j] <= 7 - 2  
B[j] <= 5
```

Values:

```
4,5
```

So:

```
2 pairs
```

For A[i] = 3

We need:

```
B[j] <= 7 - 3  
B[j] <= 4
```

Only:

```
4
```

So:

```
1 pair
```

Total:

```
3 + 2 + 1 = 6
```

Therefore:

```
count(7) = 6
```

8. How do we find the number of B elements <= X-A[i]?

Sort B.

```
B = [4,5,6]
```

Then we can use:

```
upper_bound(B.begin(), B.end(), X - A[i])
```

upper_bound gives the first element **greater than** the target.

Its position is exactly the number of elements <= target .

For example:

```
upper_bound(B.begin(), B.end(), 5)
```

returns position after 5 .

So:

```
number of elements <= 5 = 2
```

Therefore:

```
count += upper_bound(B.begin(), B.end(), X - A[i]) - B.begin();
```

9. Complexity of counting

For every element of A , we perform one binary search on B .

Therefore:

$O(N \log M)$

But there is another important trick.

The constraints say:

$$\sum \min(N, M) \leq 10^5$$

This is a strong hint.

We should always iterate over the **smaller array**.

So:

```
small array = A
large array = B
```

Sort both.

Then counting takes:

$$O(\min(N, M) \log(\max(N, M)))$$

10. Can we make counting even faster?

Yes.

Because both arrays are sorted, we don't actually need to perform a binary search for every element.

Suppose:

```
A = [1, 2, 3]
B = [4, 5, 6]
X = 7
```

For $A[i] = 1$:

```
B <= 6
```

3 elements.

For $A[i] = 2$:

```
B <= 5
```

2 elements.

For $A[i] = 3$:

```
B <= 4
```

1 element.

As $A[i]$ increases, the value:

```
X - A[i]
```

decreases.

Therefore the number of valid B elements also decreases.

We can use a **two-pointer technique**.

But `upper_bound` is already fast enough given:

$$\sum \min(N, M) \leq 10^5$$

So the simpler solution is:

```
sort
+
binary_search answer
+
upper_bound for counting
```

11. Binary search range

What values can the answer have?

Since:

```
0 <= Ai <= 10000
0 <= Bi <= 10000
```

The smallest possible sum is:

```
0 + 0 = 0
```

The largest possible sum is:

```
10000 + 10000 = 20000
```

Therefore:

```
low = 0
high = 20000
```

We binary search this range.

12. Binary search intuition

We want:

```
The smallest x such that count(x) >= K.
```

So:

```
if count(mid) >= K
```

then `mid` is **large enough**.

But maybe there is a smaller answer.

Therefore:

```
high = mid;
```

Otherwise:

```
count(mid) < K
```

means `mid` is too small.

Therefore:

```
low = mid + 1;
```

Eventually:

```
low == high
```

and that value is our answer.

13. Complete algorithm

For every test case:

Step 1

Read `A` and `B`.

Step 2

Sort both arrays.

Step 3

Make the smaller array the one we iterate over.

Step 4

Binary search x from:

```
0 to 20000
```

Step 5

For every `mid`, calculate:

$count(mid)$

using:

$$B[j] \leq mid - A[i]$$

and `upper_bound`.

Step 6

If:

```
count >= K
```

search left.

Otherwise search right.

14. Code

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;

    while (T--) {
        int N, M;
        ll K;

        cin >> N >> M >> K;

        vector<int> A(N);
        vector<int> B(M);

        for (int &x : A) cin >> x;
        for (int &x : B) cin >> x;

        // We want to iterate over the smaller array.
        if (A.size() > B.size()) {
            swap(A, B);
        }

        // Sort both arrays.
        sort(A.begin(), A.end());
        sort(B.begin(), B.end());

        int low = 0;
        int high = 20000;

        while (low < high) {
            int mid = low + (high - low) / 2;

            ll count = 0;

            for (int x : A) {
                int need = mid - x;

                // Number of elements in B <= need
```

```

        count += upper_bound(B.begin(), B.end(), need)
                  - B.begin();

        // No need to count further if we already reached K.
        if (count >= K)
            break;
    }

    if (count >= K) {
        // mid can be the answer.
        high = mid;
    } else {
        // mid is too small.
        low = mid + 1;
    }
}

cout << low << '\n';
}

return 0;
}

```

15. Let's dry-run the binary search

For:

```

A = [1,2,3]
B = [4,5,6]
K = 6

```

Answer is between:

```

0 and 20000

```

Eventually binary search reaches values around the actual answer.

Suppose:

```

mid = 6

```

We calculate:

```

A = 1
need = 6 - 1 = 5
B <= 5 -> 2

```

```

A = 2
need = 6 - 2 = 4
B <= 4 -> 1

```

```

A = 3
need = 6 - 3 = 3
B <= 3 -> 0

```

Therefore:

```

count(6) = 2 + 1 + 0
          = 3

```

But:

```

3 < K

```

So 6 is too small.

We do:

```

low = mid + 1;

```

Now suppose:

```

mid = 7

```

```
A = 1 -> B <= 6 -> 3
A = 2 -> B <= 5 -> 2
A = 3 -> B <= 4 -> 1
```

Therefore:

```
count(7) = 3 + 2 + 1
          = 6
```

And:

```
6 >= K
```

So 7 is large enough.

We do:

```
high = mid;
```

Eventually:

```
low = high = 7
```

Answer:

```
7
```

16. Why long long is VERY important

This is an easy place to make a mistake.

N and M can each be:

```
1,000,000
```

Therefore:

$$N \times M = 10^{12}$$

which is much larger than the maximum int:

$$2.147 \times 10^9$$

So K must be:

```
long long K;
```

And:

```
long long count = 0;
```

Otherwise, the count can overflow.

17. Complexity

Let:

$$S = \min(N, M)$$

and:

$$L = \max(N, M)$$

Sorting:

$$O(N \log N + M \log M)$$

For each binary-search step:

$$O(S \log L)$$

The answer range is only 0...20000, so there are approximately:

$$\log_2(20001) \approx 15$$

binary-search iterations.

Therefore the total is approximately:

$$O(N \log N + M \log M + S \log L \log 20000)$$

Since the constraint specifically guarantees:

$$\sum \min(N, M) \leq 10^5$$

this approach is efficient.

18. The core intuition to remember

Don't memorize this as just a code pattern. Remember the thought process:

```
Need K-th smallest
  ↓
Can't construct N*M elements
  ↓
Guess an answer X
  ↓
Ask: how many sums <= X?
  ↓
If count(X) >= K
  X is large enough
else
  X is too small
  ↓
count(X) is monotonic
  ↓
Binary Search on X
```

And to calculate the count:

```
A[i] + B[j] <= X
  ↓
B[j] <= X - A[i]
  ↓
Sort B
  ↓
upper_bound(B, X - A[i])
  ↓
number of valid pairs for A[i]
```

So the entire solution boils down to:

Binary search the sum value, and for each candidate sum count how many pairs can produce a sum $\leq$ that value.

This is one of the most useful patterns in competitive programming: **K-th smallest + impossible to generate all values $\rightarrow$ binary search the answer + counting function.**